

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

1 | 12-septiembre: Presentación de Asignatura y Proyecto

Contexto y Punto de Partida El inicio del segundo curso del ciclo y la presentación del módulo de Proyecto (TFG) marca un punto de inflexión en la vida de cualquier estudiante de desarrollo de software. Durante esta primera sesión del 12 de septiembre, el profesorado estableció las rúbricas de evaluación, los hitos de entrega y las expectativas de calidad. El primer y más abrumador obstáculo al que me enfrenté al salir de esa clase no fue de carácter técnico, sino analítico: ¿Qué voy a construir durante los próximos nueve meses? Es muy común caer en la tentación de elegir proyectos "cómodos" o tutorializados, como una tienda online genérica, un blog o un gestor de tareas. Sin embargo, un Trabajo de Fin de Grado debe ser la carta de presentación para el mercado laboral, y para que tenga valor, debe resolver un problema real de la sociedad.

Análisis, Reflexión y Toma de Decisiones Durante los días posteriores, adopté una mentalidad de observador y analista de sistemas en mi día a día. La inspiración surgió en el lugar menos esperado: mi propio lugar de entrenamiento. Como usuario habitual de centros deportivos y "boxes" de entrenamiento funcional, empecé a fijarme en la operativa diaria del personal de recepción y los monitores. Pude observar cómo la gestión administrativa de estos negocios —que a menudo manejan cientos de clientes— estaba anclada en metodologías de hace dos décadas. El gerente lidiaba diariamente con hojas de cálculo de Excel kilométricas que se rompían si alguien borraba una celda por error. El control de pagos se llevaba en una libreta de papel o dependía de la memoria del recepcionista. Además, el sistema de acceso a las instalaciones dependía de tarjetas magnéticas de plástico; tarjetas que los socios perdían constantemente, olvidaban en casa o se prestaban entre ellos, lo que no solo

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

suponía un agujero de seguridad, sino un gasto recurrente en la compra de nuevos plásticos para el dueño del negocio.

Conclusión y Sigüientes Pasos El sentido común me indicaba que acababa de encontrar un nicho de mercado desatendido y una oportunidad perfecta para mi proyecto. La solución tecnológica que planteé en mi mente fue **FITBOX**: una plataforma integral (SaaS - Software as a Service) 100% en la nube. Esta herramienta no solo eliminaría el papel, sino que modernizaría el acceso permitiendo a los usuarios abrir el torno del gimnasio utilizando un código QR dinámico generado en sus propios teléfonos móviles. Al finalizar la semana, ya no tenía un simple "proyecto de clase", tenía un producto con un propósito real y una propuesta de valor comercializable.

2 | 19-septiembre: Creación de Imagen Corporativa de la Empresa

Contexto y Punto de Partida En la industria actual del desarrollo de software, la excelencia del código interno (Backend) no es suficiente si la aplicación carece de una identidad visual atractiva. Los usuarios finales, e incluso los tribunales de evaluación, juzgan la calidad de un producto por su interfaz y su profesionalidad estética en los primeros cinco segundos de uso. Por tanto, antes de escribir una sola línea de código o definir bases de datos, era imperativo dotar al proyecto de una "personalidad", un nombre comercial y un manual de marca.

Análisis, Reflexión y Toma de Decisiones El primer paso fue el proceso de Naming (búsqueda de nombre comercial). Se barajaron multitud de opciones en una lluvia de ideas inicial. Nombres como "GymManager", "SportAdmin" o "TrackFit" fueron rápidamente descartados. Sonaban demasiado genéricos, aburridos y carecían del

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

impacto necesario para posicionarse como una marca moderna. El objetivo era encontrar una palabra corta, contundente, fácil de recordar y de teclear en un navegador móvil. Tras varias iteraciones, surgió la palabra **FITBOX**. Esta nomenclatura une dos conceptos fundamentales del sector: "Fit" (haciendo alusión al fitness y la salud) y "Box" (el término moderno con el que se conoce mundialmente a los centros de entrenamiento funcional o Crossfit). Con solo dos sílabas, era el nombre perfecto.

Una vez consolidado el nombre, me enfrenté al reto del diseño visual y la psicología del color. La inmensa mayoría de las aplicaciones de gestión hospitalaria o deportiva utilizan fondos blancos inmaculados con detalles en azul corporativo. Sin embargo, me puse en la piel del usuario final: un recepcionista o un monitor que pasará ocho horas diarias mirando esa pantalla en un gimnasio que, a menudo, tiene una iluminación tenue o luces de neón. Por pura ergonomía visual y sentido común, decidí que FITBOX debía ser disruptivo y nacer directamente con un diseño en **Modo Oscuro (Dark Theme)** nativo. Se definió una paleta de colores muy específica: los fondos utilizarían tonos carbón y grises muy profundos (#0f1115 y #16181d), que no emiten luz molesta a los ojos. Para romper la monotonía y guiar la atención del usuario, se eligió un color de contraste agresivo: un rojo carmesí vibrante (#dc2626). Este rojo no es casual; en la psicología del color deportivo, el rojo evoca energía, elevación del ritmo cardíaco, esfuerzo, pasión y poder.

Conclusión y Sigüientes Pasos El trabajo de la semana concluyó con el diseño de un logotipo minimalista en formato vectorial (SVG). Al trabajar con gráficos vectoriales, garanticé matemáticamente que el logotipo jamás se pixelaría, manteniendo una nitidez absoluta tanto en la pequeña pantalla de un smartphone como en un monitor 4K en la recepción del gimnasio. FITBOX ya tenía una identidad corporativa premium, lista para respaldar la arquitectura técnica que estaba por venir.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

3 | 26-septiembre: Elaboración de Contrato y Recogida de Necesidades

Contexto y Punto de Partida Uno de los mayores peligros a los que se enfrenta un desarrollador junior se conoce en la industria como Scope Creep (o "síndrome del lavadero"). Este fenómeno ocurre cuando, al no tener límites claros, se empiezan a añadir "pequeñas" funcionalidades al proyecto de forma descontrolada: un día se decide añadir un chat, al siguiente una tienda de suplementos, y al otro una red social interna. El resultado es un proyecto inabarcable que agota el tiempo del programador y termina fracasando. Para evitar este destino, debía actuar como una consultora de software profesional y formalizar un "contrato" de alcance.

Análisis, Reflexión y Toma de Decisiones Dado el carácter académico del TFG, simulé una serie de reuniones de levantamiento de requisitos basándome en conversaciones reales previas con dueños de centros deportivos. El objetivo de esta semana era separar quirúrgicamente lo que era una "necesidad absoluta" de lo que era un "capricho innecesario". Durante este ejercicio de empatía empresarial, identifiqué y documenté los verdaderos "Puntos de Dolor" (Pain points) del cliente:

1. La gestión de morosidad: El gerente pierde horas cruzando datos bancarios para saber quién le debe la mensualidad.
2. El caos de los aforos: En las clases populares (ej. Spinning a las 19:00h), los socios reservan por WhatsApp, a veces no asisten dejando plazas vacías, o asisten de más, generando conflictos por la falta de bicicletas.
3. El coste de acceso: La impresión y reposición de tarjetas físicas supone un gasto de cientos de euros anuales que no aporta valor.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

4. La comunicación de incidencias: Las máquinas de musculación se estropean y el gerente tarda días en enterarse porque los monitores olvidan avisarle.

Durante esta , surgieron ideas tentadoras y espectaculares, como integrar rutinas de entrenamiento generadas por Inteligencia Artificial o pasarelas de pago con criptomonedas. Sin embargo, la madurez profesional exigió descartarlas rotundamente para la versión 1.0 (MVP - Producto Mínimo Viable).

Conclusión y Sigüientes Pasos Se redactó un documento formal simulando un contrato comercial vinculante. En él se estipuló que el desarrollo del TFG se centraría **exclusivamente** en resolver de forma perfecta y pulida esos cuatro problemas fundamentales: pagos, aforos, accesos QR y mantenimiento. Firmar este documento (aunque fuera de manera figurada) me proporcionó una hoja de ruta blindada, garantizando que no me desviaría del objetivo principal en los meses venideros.

4 | 03-octubre: Definición de Requisitos (Funcionales/No Funcionales)

Contexto y Punto de Partida Con el "contrato de alcance" cerrado, el proyecto estaba conceptualmente definido. Sin embargo, decir "quiero que los clientes no puedan reservar si la clase está llena" es una frase en lenguaje humano, válida para una reunión comercial, pero inútil para un ingeniero de software. El objetivo de esta semana era someter esas necesidades de negocio a un proceso de traducción técnica, redactando el Documento de Especificación de Requisitos.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Análisis, Reflexión y Toma de Decisiones Dediqué la semana a diseccionar la aplicación y categorizar exhaustivamente sus especificaciones en dos grandes bloques innegociables:

- **Requisitos Funcionales (El "Qué" debe hacer el sistema para existir):**
 - Se estableció la obligatoriedad de programar un sistema RBAC (Control de Acceso Basado en Roles) con tres niveles jerárquicos: Administrador (acceso total), Monitor (acceso limitado a sus clases) y Socio (acceso restringido a su información personal).
 - El sistema deberá contar con un algoritmo capaz de generar, en tiempo de ejecución, un Código QR único por socio que se actualice dinámicamente para evitar fraudes (como capturas de pantalla).
 - Se deberá programar un sistema CRUD (Create, Read, Update, Delete) para la gestión completa del inventario de máquinas del centro.
 - La lógica matemática de reservas deberá ejecutarse en tiempo real: el sistema calculará constantemente el aforo máximo de una sala restando los asistentes actuales, y si el resultado es cero, deshabilitará automáticamente cualquier interacción de reserva en la interfaz.
- **Requisitos No Funcionales (El "Cómo" debe comportarse el sistema para ser utilizable):**
 - Paradigma Mobile First: Este fue el requisito más dictado por el sentido común. Un usuario no lleva un ordenador portátil en la mochila del gimnasio; interactúa exclusivamente con su smartphone. Por tanto, establecí que el 100% de la interfaz pública debía diseñarse, maquetarse y probarse primero en pantallas de 320 píxeles de ancho (móviles antiguos) antes de adaptarse a monitores de escritorio.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

- Seguridad y Privacidad: Al tratar con datos sensibles (correos electrónicos, nombres completos, historial de pagos), el sistema debe cumplir teóricamente con el RGPD. Se estableció como requisito inquebrantable que las contraseñas jamás viajarán ni se almacenarán en las bases de datos en texto plano, debiendo utilizar algoritmos de encriptación y hashing avanzados (como bcrypt).
- Rendimiento (Performance): Se estableció que las transiciones entre pantallas deben ocurrir en menos de 1 segundo, eliminando las recargas de página en blanco, simulando la velocidad de una aplicación móvil nativa.

Conclusión y Sigüientes Pasos Al finalizar la redacción de este documento, FITBOX dejó de ser una simple "buena idea" para convertirse en un plano arquitectónico técnico sólido y auditable. Las reglas del juego, los límites del sistema y las exigencias de calidad estaban perfectamente documentadas y listas para ser plasmadas visualmente.

5 | 10-octubre: Desarrollo de las Interfaces Gráficas

Contexto y Punto de Partida En el mundo del desarrollo frontend, empezar a programar código HTML y CSS sin tener un plano visual previo es el equivalente a intentar construir una casa empezando por el tejado. Te obliga a rehacer el trabajo una y otra vez, moviendo botones "a ciegas" hasta que algo encaje, lo cual consume cientos de horas inútilmente. Por ello, esta semana me alejé por completo de los editores de código para centrarme exclusivamente en el diseño de la Interfaz de Usuario (UI) y la Experiencia de Usuario (UX).

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Análisis, Reflexión y Toma de Decisiones El objetivo era crear Wireframes (bocetos estructurales) y prototipos de alta fidelidad. Para lograrlo con éxito, tuve que aplicar una profunda empatía hacia los futuros usuarios de FITBOX, entendiendo que cada rol tiene necesidades radicalmente opuestas.

Para el diseño del **Dashboard del Socio**, apliqué la Ley de Fitts (un principio de ergonomía que dicta que el tiempo para alcanzar un objetivo depende de su tamaño y distancia). Imaginé a un cliente llegando al gimnasio a las 6:30 de la mañana, adormilado y con prisas por entrar al turno. Ese usuario no quiere navegar por menús desplegables complejos. Por consiguiente, tomé la decisión de que la pantalla de inicio del cliente fuera extremadamente minimalista: el 50% de la pantalla estaría ocupado por un botón gigante que, al pulsarse, mostraría su Código QR. Además, se ideó un sistema de comunicación pasiva mediante un "semáforo" visual; si el borde de su foto de perfil es rojo, el usuario sabe intuitiva e instantáneamente que tiene un pago pendiente, sin necesidad de leer ningún texto de advertencia.

Por el contrario, para el **Dashboard del Administrador**, la necesidad era la opuesta. El gerente del gimnasio está sentado en una silla, tranquilo, frente a un monitor panorámico de 24 pulgadas, y necesita cruzar mucha información simultánea. Para él, diseñé un layout clásico de software ERP de alta gama: una barra lateral fija (Sidebar) a la izquierda de color oscuro para la navegación principal, dejando un inmenso lienzo central blanco/gris claro para mostrar tablas expansivas de facturación, listas de socios y calendarios semanales.

Conclusión y Sigüientes Pasos Estos diseños no fueron simples dibujos artísticos para rellenar un documento. Establecieron la base matemática de los componentes que más tarde tendría que programar en React. Definieron los márgenes, los tamaños

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

tipográficos y la jerarquía visual de la aplicación. El ahorro de tiempo y la reducción de estrés que este trabajo previo supondría en las semanas de programación de marzo serían incalculables.

6 | 17-octubre: Desarrollo de la Estructura de la Base de Datos

Contexto y Punto de Partida Cualquier aplicación de gestión, por muy bonita e intuitiva que sea por fuera, se reduce en su núcleo a una sola cosa: mover, almacenar y modificar información. Habiendo cerrado el diseño visual, tocaba ponerse el casco de Ingeniero de Datos y bajar a la sala de máquinas del proyecto. El primer gran dilema técnico del curso se presentó aquí: ¿Qué paradigma de almacenamiento de base de datos debía utilizar para FITBOX?

Análisis, Reflexión y Toma de Decisiones En la actualidad tecnológica, las bases de datos NoSQL (basadas en documentos, como MongoDB o Firebase Firestore) gozan de una inmensa popularidad por su flexibilidad y facilidad de aprendizaje. Permiten guardar objetos JSON directamente sin preocuparse por un esquema rígido. Sin embargo, en la ingeniería de software profesional, las decisiones no se toman por modas pasajeras, sino por la idoneidad frente al problema a resolver.

Me senté a analizar meticulosamente la naturaleza de los datos que se manejan en un gimnasio. Son datos rígidamente estructurados y con altísima interdependencia. Veamos un ejemplo: Un "Usuario" (Socio) realiza una "Reserva" para una "Clase" específica, la cual es impartida por otro "Usuario" (Monitor). Además, el primer usuario realiza "Pagos" mensuales asociados a su cuenta. Las bases de datos NoSQL son magníficas para almacenar grandes volúmenes de datos aislados (como los

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

comentarios de un blog o los logs de error de un servidor), pero sufren terriblemente cuando se trata de relacionar información. Si usara NoSQL, relacionar a un usuario con sus clases y sus pagos requeriría hacer múltiples peticiones de red al servidor y unir los datos manualmente mediante código en el teléfono del cliente, lo que destrozaría el rendimiento de la aplicación móvil y consumiría su batería.

El sentido común y los principios básicos de la ingeniería exigían una integridad referencial estricta. Si un monitor cancela una clase, las reservas de todos los socios asociadas a esa clase deben desaparecer automáticamente en cascada para mantener la coherencia. Por consiguiente, tomé la decisión técnica rotunda e inamovible de utilizar un motor de **Base de Datos Relacional (SQL)**. Concretamente, se perfiló el uso de PostgreSQL, reconocido mundialmente por su robustez, su tipado estricto de datos y su capacidad para realizar uniones (JOINS) extremadamente complejas en milisegundos directamente en la capa del servidor.

Conclusión y Sigüientes Pasos Con el paradigma relacional firmemente elegido, procedí a listar de forma teórica las entidades abstractas que darían vida al ecosistema del gimnasio: Usuarios, Roles, Clases, Máquinas, Reservas y Pagos. El terreno estaba preparado para el siguiente paso: someter estas entidades a las rigurosas reglas matemáticas de normalización y trazar las relaciones entre ellas.

7 | 24-octubre: Definición de Modelo Relacional de la Base de Datos

Contexto y Punto de Partida Definidas las tablas abstractas la semana anterior, el objetivo ahora era aplicar la teoría matemática de bases de datos relacionales. Había que crear el Diagrama Entidad-Relación y someter todas las tablas al proceso de

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Normalización (garantizando el cumplimiento de la Primera, Segunda y Tercera Forma Normal). El objetivo de este riguroso proceso es asegurar que la base de datos nunca sufra anomalías de actualización; es decir, si un gimnasio cambia el nombre de la clase de "Spinning" a "Ciclo Indoor", este cambio debe reflejarse instantáneamente en todo el sistema sin tener que actualizar miles de registros de reservas duplicadas.

Análisis, Reflexión y Toma de Decisiones Durante la confección del esquema, me enfrenté a varias decisiones arquitectónicas críticas que definirían la seguridad a largo plazo del sistema.

La primera fue el diseño de la tabla principal: usuarios. Tradicionalmente, al crear una tabla, los desarrolladores configuran la Clave Primaria (PK) como un número entero autoincremental (ID 1, ID 2, ID 3...). Sin embargo, me detuve a pensar en las implicaciones de seguridad en un entorno real. Si un usuario entra en su perfil y ve en la URL que es el "socio_id=150", y a la semana siguiente su amigo se registra y es el "socio_id=155", la competencia de gimnasios de la ciudad puede deducir exactamente que están captando 5 clientes a la semana. Peor aún, un atacante informático podría crear un pequeño programa (script) que hiciera un bucle raspando las URLs (/usuario/1, /usuario/2) para robar indiscriminadamente toda la base de datos de clientes. Para bloquear esta vulnerabilidad desde el diseño, decidí que la Clave Primaria de la tabla usuarios sería un **UUID** (Identificador Único Universal). Un UUID es una cadena alfanumérica criptográfica de 36 caracteres (ej. 550e8400-e29b-41d4-a716-446655440000) completamente indescifrable y no secuencial, añadiendo una capa vital de ciberseguridad pasiva al sistema.

El segundo gran reto fue modelar la inscripción de los socios a las clases. La cardinalidad era compleja pero evidente: Un socio asiste a muchas clases, y una clase

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

tiene a muchos socios apuntados. Estaba ante una relación de Muchos a Muchos (N:M). En el paradigma relacional, esto es un problema que se resuelve "rompiendo" la relación mediante una tabla intermedia. Creé la tabla reservas, que actuaría como puente. Esta tabla contendría la Clave Foránea del usuario, la Clave Foránea de la clase y, lo que resultó ser una decisión crucial, un campo fecha_reserva de tipo Timestamp. Añadir este campo temporal me permitiría registrar exactamente en qué milisegundo un usuario hizo clic en "Reservar", un dato absolutamente vital si en el futuro el gimnasio quisiera implementar un sistema justo de "Listas de Espera" por orden de llegada.

Conclusión y Sigüientes Pasos El diseño final arrojó un esquema relacional limpio, elegantemente estructurado, fuertemente tipado e interconectado mediante claves foráneas con políticas de borrado en cascada. El "cerebro" y la memoria a largo plazo de FITBOX estaban contruidos teóricamente, listos para ser sometidos al escrutinio y validación externa en la siguiente semana.

8 | 31-octubre: Presentación a la Empresa (Interfaces y BD)

Contexto y Punto de Partida Existe un fenómeno psicológico y técnico muy peligroso en el desarrollo de software conocido en la industria como la "visión de túnel". Cuando un programador o arquitecto de software pasa demasiadas semanas trabajando en aislamiento total sobre un proyecto, tiende a enamorarse de su propia solución. El cerebro se acomoda a la estructura que ha creado y deja de percibir fallos lógicos graves que resultarían evidentes y flagrantes para cualquier persona externa que observe el proyecto por primera vez. Para romper esta dinámica, era imprescindible

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

realizar una parada técnica y someter el proyecto a una auditoría simulada para validar que el rumbo era el correcto antes de empezar a programar.

Análisis, Reflexión y Toma de Decisiones Preparé una simulación de reunión comercial y técnica con el tutor del proyecto, quien adoptó el papel dual de "cliente final" (el dueño del gimnasio) y de auditor de sistemas. Durante esta sesión, expuse detalladamente los wireframes visuales de las pantallas y expliqué el flujo de datos apoyándome en el Diagrama del Modelo Entidad-Relación de la base de datos que había finalizado la semana anterior.

Esta sesión de feedback resultó ser uno de los momentos de mayor aprendizaje y valor añadido de todo el primer trimestre. Mientras yo explicaba con cierto orgullo técnico cómo mi tabla de pagos registraba meticulosamente la cantidad exacta de dinero abonada por el cliente y el identificador del mes, el "cliente" me interrumpió con una pregunta basada puramente en el sentido común empresarial: "Si un cliente problemático llega a la puerta del gimnasio, pasa su móvil por el torno de entrada un día 5 de noviembre, y resulta que su pago bancario ha sido devuelto o rechazado, ¿cómo sabe tu sistema, en cuestión de milisegundos, si debe abrirle la puerta o bloquearle el paso?"

La pregunta fue un jarro de agua fría. Al revisar mentalmente mi modelo de base de datos, me di cuenta de un error arquitectónico: mi tabla obligaba al sistema a buscar todo el historial de pagos de un usuario, cruzar las fechas de cobro con la fecha actual del sistema, y deducir matemáticamente si el mes en curso estaba cubierto. Este proceso de cálculo en tiempo real, multiplicado por cientos de usuarios entrando a las horas punta, sobrecargaría el servidor y retrasaría la apertura del torno. Tomé nota inmediata de la corrección. Esa misma tarde, iteré y modifiqué el modelo de la base de

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

datos, añadiendo campos explícitos de control de estado (estado_pago: Pendiente, Pagado, Expirado). Al tener el estado guardado directamente en una columna, la consulta a la base de datos al pasar por el turno tomaría apenas unos milisegundos, y además facilitaría enormemente la programación de los "semáforos" visuales rojo/verde que había diseñado para la interfaz del usuario.

Conclusión y Sigüientes Pasos Esta demostró empíricamente por qué las metodologías de desarrollo ágiles e iterativas funcionan mejor que las tradicionales en cascada. Detectar un fallo arquitectónico lógico en la base de datos a finales de octubre me costó apenas una tarde de modificar cajas y flechas en un diagrama. Haber detectado ese mismo fallo lógico en el mes de mayo, con el servidor en producción y miles de líneas de código y consultas SQL ya escritas y entrelazadas, habría supuesto un desastre técnico casi irrecuperable. Con el modelo de datos finalmente validado y pulido frente a la realidad del negocio, el proyecto estaba maduro para entrar en la elección de herramientas de programación definitivas.

9 | 07-noviembre: Elección de Tecnologías a Utilizar

Contexto y Punto de Partida Con los planos visuales del edificio terminados y la estructura de la base de datos auditada, llegaba uno de los momentos más delicados y determinantes para cualquier ingeniero: la elección de los "materiales de construcción", es decir, el Stack Tecnológico. El ecosistema del desarrollo web moderno es brutalmente dinámico; sufre mutaciones, actualizaciones y cambios de paradigma cada pocos meses. Elegir una tecnología obsoleta o poco demandada significaría que el proyecto del TFG nacería "muerto" tecnológicamente, no sería escalable y, lo que es

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

peor, no aportaría valor real a mi currículum ni a mi futuro perfil profesional en el mercado laboral.

Análisis, Reflexión y Toma de Decisiones Invertí esta semana en realizar una profunda investigación de mercado. Analicé decenas de ofertas de empleo para perfiles Junior/Mid en plataformas como LinkedIn e InfoJobs, y revisé las encuestas anuales de desarrolladores (como la de StackOverflow) para entender hacia dónde se dirigía la industria. Basándome en datos y no en preferencias personales, dividí la arquitectura tecnológica en tres grandes pilares innegociables:

1. **Frontend (La capa de presentación y lógica de cliente):** Descarté de forma inmediata e instintiva la idea de crear la web utilizando plantillas estáticas de HTML/PHP clásico enlazadas con jQuery. El modelo tradicional, donde el navegador se queda en blanco y recarga toda la página cada vez que pulsas un enlace, habría destrozado la Experiencia de Usuario (UX) en los teléfonos móviles de los clientes del gimnasio. Era un requisito innegociable construir una SPA (Single Page Application), donde la web carga una sola vez y reacciona de forma instantánea. La decisión lógica y avalada por la industria fue utilizar **React**, la potente librería creada y mantenida por Facebook, por ser el estándar absoluto del mercado actual. Además, decidí acoplar React con **TypeScript**. Aunque TypeScript tiene una curva de aprendizaje más dura al obligar a tipar todas las variables, era una red de seguridad indispensable para evitar que un error tonto (como intentar sumar un texto a un número) provocara que la aplicación del gimnasio se colgara en tiempo de ejecución (Run-time).
2. **Estilos (La capa de diseño y maquetación):** Se descartó categóricamente el uso de Bootstrap. Aunque es fácil de usar, sus componentes prefabricados hacen que todas las páginas web de internet parezcan clones sin personalidad.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Yo necesitaba que el "Modo Oscuro" de FITBOX brillara con luz propia. Opté por la vanguardia del diseño moderno: **Tailwind CSS**. Un revolucionario framework basado en "clases de utilidad". En lugar de tener que saltar constantemente entre mi código de React y un archivo .css externo, interminable y confuso, Tailwind me permitiría esculpir el diseño, los colores y las sombras escribiendo clases cortas directamente sobre los componentes de la interfaz.

3. **Backend (El servidor, la autenticación y la base de datos):** Esta fue, con diferencia, la decisión estratégica más crítica para la viabilidad del TFG. Programar una API RESTful desde cero en Node.js (con Express), configurar los puertos de red, programar a mano el enrutamiento, establecer los middlewares de seguridad para proteger los endpoints, configurar la validación de Tokens JWT, programar los algoritmos matemáticos de encriptación bcrypt para las contraseñas, y finalmente, levantar y mantener un servidor físico de PostgreSQL... me habría llevado, siendo optimista, 4 o 5 meses de trabajo ininterrumpido. Un trabajo de infraestructura pesado que, a los ojos del dueño del gimnasio, no aporta ningún valor directo a su negocio. Aplicando un estricto sentido común y priorizando la entrega de valor, tomé la decisión de delegar toda esta pesada capa de infraestructura. Decidí que FITBOX utilizaría una arquitectura Backend-as-a-Service (BaaS), apoyándome en plataformas en la nube de nivel empresarial que ya tienen todo el sistema de servidores, bases de datos y seguridad de login resuelto y escalado. Esta decisión abriría la puerta a estudiar y enfrentar a titanes como Firebase y Supabase en los meses posteriores.

Conclusión y Sigüientes Pasos La cristalización del Stack tecnológico de FITBOX (React + TypeScript + Tailwind CSS + Arquitectura BaaS) no fue fruto de la casualidad

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

ni de la costumbre académica. Fue una decisión calculada que posicionaba el TFG como un proyecto vanguardista, altamente escalable y perfectamente alineado con las exigencias tecnológicas de las grandes empresas de software del panorama actual. El ecosistema estaba cerrado; el siguiente paso era comenzar a documentarlo todo.

10 | 14-noviembre: Estructuración Inicial de Documentación

Contexto y Punto de Partida En el mundo del desarrollo de software profesional, existe un axioma irrefutable: un código brillante y perfectamente optimizado que carece de documentación es, a efectos prácticos, un software inútil e inmantenible a largo plazo. La industria exige que todo proyecto cuente con una base documental sólida que permita a futuros desarrolladores entender la arquitectura, y a los usuarios finales comprender cómo manejar la herramienta. Esta semana de mediados de noviembre marcó el fin de la etapa creativa y el inicio de la pesada, pero académicamente obligatoria, carga burocrática y documental del TFG.

Análisis, Reflexión y Toma de Decisiones El principal error que cometen la mayoría de los desarrolladores junior (y muchos estudiantes) al documentar sus proyectos es la falta de empatía hacia el lector. Suelen volcar todo su conocimiento en un único y gigantesco archivo PDF de 150 páginas donde mezclan, sin pudor, instrucciones técnicas sobre cómo configurar los puertos de la base de datos, junto con tutoriales básicos sobre cómo hacer clic en el botón de "Recuperar contraseña". Esto carece de todo sentido común, ya que la audiencia de un software no es homogénea.

Para resolver este problema de comunicación, decidí aplicar un patrón de diseño documental basado en la "separación de intereses" (Separation of Concerns). Dividí y

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

estructuré toda la documentación del proyecto aislando a los lectores objetivos en cuatro pilares fundamentales y altamente especializados:

1. **Manual del Proyecto:** (El documento evolutivo). Su objetivo es puramente académico y metodológico. Está destinado al tutor y al tribunal de evaluación. En él se relataría la historia, la evolución semanal, los problemas encontrados y la justificación de cada decisión tomada durante los 9 meses de curso.
2. **Manual Técnico:** El documento de ingeniería pura. Orientado estrictamente a otros desarrolladores de software, arquitectos de sistemas o auditores. Este manual se despojaría de lenguaje coloquial para centrarse en esquemas de red, el modelo entidad-relación detallado, los algoritmos de encriptación utilizados y las políticas de seguridad en el servidor.
3. **Manual de Usuario:** El documento del cliente. Redactado con un lenguaje amigable, directo, empático y altamente visual (apoyado en capturas de pantalla). Su objetivo es que un recepcionista de gimnasio, sin ningún tipo de conocimiento en informática, sea capaz de operar FITBOX desde el primer día sin frustraciones.
4. **Manual de Despliegue:** El documento de infraestructura. Orientado al Administrador de Sistemas (SysAdmin). Una guía espartana, paso a paso, con los comandos exactos de terminal necesarios para clonar el repositorio, inyectar las variables de entorno secretas y publicar la web en servidores de producción accesibles a nivel mundial.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Contexto y Punto de Partida Con los esqueletos de los cuatro documentos maestros ya creados, el reto de esta semana era diseñar los índices detallados. Un error muy común en la ingeniería de software es escribir documentación de "relleno" para alcanzar un número mínimo de páginas, resultando en textos densos, repetitivos y carentes de utilidad práctica. El objetivo primordial aquí era aplicar ingeniería inversa: pensar en qué problemas tendrían los futuros lectores de estos manuales y estructurar los apartados como respuestas directas a esos problemas. No se trataba de escribir por escribir, sino de crear una herramienta de consulta eficiente.

Análisis, Reflexión y Toma de Decisiones Comencé por el **Manual Técnico**, cuyo público objetivo son otros programadores, arquitectos de sistemas o el tribunal evaluador. Me senté a reflexionar sobre qué información me exigiría a mí mismo si una empresa me contratara para mantener el código de FITBOX dentro de tres años. La respuesta fue clara: no necesito que me expliquen qué es un bucle for, necesito saber la arquitectura. Por ello, definí apartados estrictos y al grano:

1. Estructura de enrutamiento de la SPA (Single Page Application) y protección de rutas privadas.
2. Diccionario de datos y diagrama Entidad-Relación de PostgreSQL.
3. Gestión de variables de entorno (dónde se guardan las claves secretas de la base de datos).
4. Configuración del motor de estilos y el Design System.

A continuación, pasé al **Manual de Usuario**, cambiando radicalmente mi mentalidad. Este documento lo va a leer el gerente del gimnasio o un recepcionista sin conocimientos informáticos. Explicarles el menú superior o qué hace cada botón individualmente es aburrido y poco didáctico. Aplicando empatía y sentido común,

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

decidí que este manual se estructuraría a través de Workflows (Flujos de Trabajo Reales). Los capítulos no se llamarían "Pantalla de Clases", sino que se titularían como problemas a resolver: "¿Cómo dar de alta a un nuevo socio que acaba de llegar a la recepción?", "¿Cómo pasar lista de asistencia en la clase de spinning de las 19:00h?", o "¿Qué pasos seguir si un monitor detecta una máquina rota?".

Conclusión y Sigüientes Pasos Al finalizar la semana, disponía de índices quirúrgicamente precisos. Tener una estructura basada en "Casos de Uso" (Use Cases) y resolución de problemas (Troubleshooting) garantizaba que la redacción posterior fluiría de manera natural y que el resultado final de los manuales de FITBOX sería de un nivel profesional sobresaliente.

12 | 05-diciembre: Desarrollo Inicial de Manuales

Contexto y Punto de Partida Con los índices cerrados y aprobados, llegó el momento de enfrentarse a la página en blanco y comenzar la redacción masiva de los contenidos. Sin embargo, antes de teclear la primera palabra, debía tomar una decisión técnica sobre el formato y la herramienta de redacción. Tradicionalmente, la mayoría de estudiantes utilizan procesadores de texto como Microsoft Word para esto. Pero, ¿es esa realmente la mejor opción para un proyecto moderno de ingeniería de software?

Análisis, Reflexión y Toma de Decisiones Analicé los problemas de usar herramientas ofimáticas clásicas: los archivos .docx se desmaquetan al abrirlos en distintos ordenadores, requieren envío por correo electrónico o subidas manuales a la nube, y complican terriblemente el control de versiones (generando archivos como "Manual_Final_V3_Definitivo.docx"). El sentido común de un desarrollador exigía

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

abandonar las herramientas de oficina y utilizar las herramientas de la industria tecnológica.

Tomé la firme decisión de que el 100% de la documentación técnica y de usuario se redactaría utilizando **Markdown (.md)**. Markdown es un lenguaje de marcado ligero que permite escribir texto plano aplicando formato (negritas, cursivas, tablas, fragmentos de código) mediante caracteres especiales, y es el estándar absoluto en plataformas de repositorios como GitHub. Esta decisión estratégica tuvo tres beneficios inmediatos e incalculables:

1. Integración con el Código: La documentación viviría físicamente dentro del propio repositorio del proyecto (en una carpeta llamada /docs).
2. Control de Versiones: Al estar en archivos .md, cada vez que yo hiciera un commit (guardado) de una nueva pantalla en React, podría hacer un commit del manual actualizándolo simultáneamente. El código y la documentación evolucionarían de la mano, sin dess.
3. Diagramas Dinámicos: Markdown soporta de forma nativa la sintaxis Mermaid.js, lo que me permitiría dibujar el diagrama Entidad-Relación de la base de datos escribiendo simplemente líneas de texto, sin depender de imágenes externas que pierden calidad al ampliarse.

Conclusión y Sigüientes Pasos Esta semana supuso un esfuerzo titánico de mecanografía y redacción, traduciendo toda la arquitectura teórica conceptualizada en octubre y noviembre a los archivos Markdown definitivos. El resultado fue una base documental que no solo era completa y exhaustiva, sino que estaba integrada nativamente en el ecosistema de desarrollo, ofreciendo una carta de presentación impecable y altamente profesional en el repositorio de GitHub.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

13 | 12-diciembre: Opciones de Despliegue de Aplicativos

Contexto y Punto de Partida A medida que se acercaban las vacaciones de Navidad y el cierre del trabajo puramente teórico del primer trimestre, surgió una pregunta de infraestructura crucial. Un software de gestión, por muy bien programado que esté en el ordenador del desarrollador (localhost), es completamente inútil si no es accesible mundialmente a través de Internet. El objetivo de esta semana era investigar, comparar y decidir cómo y dónde se iba a alojar FITBOX cuando estuviera terminado.

Análisis, Reflexión y Toma de Decisiones En el panorama actual de alojamiento web (Hosting), existen multitud de caminos. Por un lado, tenemos los servidores compartidos tradicionales (Hosting por cPanel) y los Servidores Privados Virtuales (VPS). Por otro lado, tenemos la vanguardia tecnológica: el Edge Computing y los despliegues sin servidor (Serverless).

Realicé un análisis de viabilidad técnica cruzando las opciones con la naturaleza de FITBOX. Mi proyecto no es una página web tradicional de PHP o WordPress que genera el HTML en el servidor en cada clic. FITBOX es una Single Page Application (SPA) programada en React. Esto significa que el servidor solo necesita enviar un archivo index.html básico y un paquete compilado de JavaScript una única vez; a partir de ahí, el teléfono móvil del usuario se encarga de todo el renderizado visual y la navegación. Alquilar un VPS completo en Linux, configurar un servidor Nginx o Apache, gestionar los certificados SSL (HTTPS) a mano mediante Certbot y configurar firewalls, suponía un trabajo de Administración de Sistemas (SysAdmin) colosal. Aunque es un conocimiento valioso, dedicar semanas a configurar servidores me restaría tiempo

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

crítico para programar la lógica del gimnasio, que es donde realmente reside el valor del TFG.

El sentido común dictaba que debía buscar una solución automatizada, gratuita para la de desarrollo, y orientada específicamente a proyectos estáticos de Frontend.

14 | 19-diciembre: Pruebas de Despliegue en Entorno Local (TomCat)

Contexto y Punto de Partida Antes del parón lectivo de diciembre, y siguiendo las metodologías aprendidas en los módulos de Desarrollo Web en Entorno Servidor, era un requisito indispensable realizar pruebas de despliegue en un entorno controlado y local. El objetivo era comprender los fundamentos de cómo un servidor HTTP atiende las peticiones y entrega los archivos al cliente, utilizando para ello el servidor Apache Tomcat, un estándar clásico en la educación informática.

Análisis, Reflexión y Toma de Decisiones Procedí a descargar e instalar Apache Tomcat en mi máquina de desarrollo. Creé una versión compilada básica de prueba (un build) de una aplicación web y deposité los archivos estáticos en el directorio webapps del servidor. Arranqué el servicio de Tomcat y accedí a través del puerto 8080 en mi navegador.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Inicialmente, la aplicación cargó correctamente. Sin embargo, el choque con la realidad y los problemas arquitectónicos no tardaron en aparecer. FITBOX, al ser una Single Page Application (SPA), utiliza un sistema de enrutamiento del lado del cliente (React Router). Esto significa que si el usuario navega a localhost:8080/dashboard, React intercepta la URL y pinta la pantalla correspondiente sin pedir nada al servidor. El problema fatal surgió al intentar recargar la página (pulsar F5) estando en la ruta /dashboard. Apache Tomcat, siendo un servidor tradicional, recibió la petición y buscó físicamente una carpeta llamada "dashboard" con un archivo index.html en su interior. Al no encontrarla (porque en React solo existe un único index.html raíz), Tomcat devolvió un fatídico Error 404 (Not Found).

Para solucionar esto en Tomcat, tendría que haber configurado complejas reglas de reescritura de URLs (URL Rewriting o Fallback) en archivos de configuración del servidor, forzando a que cualquier petición devolviera siempre el index.html principal.

Conclusión y Sigüientes Pasos Esta prueba práctica fue inmensamente valiosa porque demostró empíricamente una lección de arquitectura: los servidores como Apache Tomcat están optimizados y diseñados para servir aplicaciones monolíticas tradicionales (como JSP, Servlets o Spring Boot), pero resultan pesados, ineficientes y problemáticos para servir los archivos hiper-optimizados que genera una arquitectura moderna de React. Quedó patente y documentado que este no era el camino correcto para el ecosistema de JavaScript, dejando la puerta abierta a buscar plataformas Serverless a la vuelta de las vacaciones.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Contexto y Punto de Partida De vuelta de las vacaciones de Navidad, el proyecto encaraba su recta final hacia la programación pura. Sin embargo, quedaba resolver el escollo de la infraestructura que había dejado pendiente el fracaso relativo del despliegue en Tomcat. Necesitaba una plataforma de alojamiento en la nube (Hosting) moderna, que entendiera nativamente cómo funciona una SPA (Single Page Application) y que eliminara la carga administrativa del servidor.

Análisis, Reflexión y Toma de Decisiones Tras revisar la documentación del ecosistema Frontend, me decanté por realizar pruebas con **Vercel**, la plataforma líder actual en Edge Computing y creadora del famoso framework Next.js.

El proceso de prueba fue revelador y representó un salto cualitativo en mi forma de entender la ingeniería de software moderna (DevOps). Me creé una cuenta en Vercel y, en lugar de subir archivos por FTP (un protocolo obsoleto, lento y propenso a errores humanos), Vercel me pidió simplemente que le diera permisos para leer mi repositorio de GitHub. Al enlazar el repositorio del TFG, ocurrió la "magia" de la Integración y Despliegue Continuo (CI/CD):

1. Vercel detectó automáticamente que el proyecto estaba construido sobre React/Vite.
2. Descargó todas las dependencias (npm install) en sus propios servidores en la nube.
3. Ejecutó el comando de compilación optimizada.
4. Publicó la aplicación en una red de distribución global (CDN), asignándole automáticamente una URL pública y un certificado de seguridad SSL (HTTPS) de grado bancario.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Todo este proceso ocurrió en menos de 45 segundos y sin que yo tuviera que configurar un solo puerto de servidor. Además, comprobé que Vercel soluciona nativamente el "Error 404" que me dio Tomcat; la plataforma está preconfigurada para enrutar cualquier URL hacia el index.html de React, garantizando que la navegación directa a rutas como /reservas funcionara perfectamente.

Conclusión y Sigüientes Pasos El experimento fue un éxito rotundo. Se tomó la decisión arquitectónica definitiva de que Vercel sería el entorno de Producción de FITBOX. Esta decisión eliminó de un plumazo todas mis preocupaciones sobre certificados de seguridad, caídas del servidor y transferencias de archivos. Con el problema de la infraestructura completamente resuelto y documentado, mi mente ya estaba libre para concentrarse al 100% en el código fuente.

16 | 16-enero: Investigación y Evaluación del Stack Tecnológico (React, Vite, Tailwind v4)

Contexto y Punto de Partida Con el entorno de producción resuelto, la lógica dictaba que el siguiente paso era abrir el editor de código y empezar a programar. Sin embargo, el sentido común y la experiencia en el sector tecnológico imponen cautela. El ecosistema de JavaScript evoluciona a un ritmo tan vertiginoso que las herramientas que eran el estándar de la industria en septiembre, cuando ideé el proyecto, podían haber quedado obsoletas en enero. Era obligatorio detenerse a estudiar las últimas versiones (Releases) del stack tecnológico elegido.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Análisis, Reflexión y Toma de Decisiones Dediqué la semana a leer exhaustivamente la documentación oficial y los "Changelogs" (notas de parche) de las herramientas que conformarían el núcleo de FITBOX:

- El adiós a Webpack y la era de Vite:** Durante años, el estándar para inicializar un proyecto de React era la herramienta Create-React-App (CRA), impulsada por el empaquetador Webpack. Al investigar, descubrí que la propia comunidad de React había declarado CRA como oficialmente "muerto" y descontinuado (Deprecated). Webpack, al ser lento y estar escrito en JavaScript puro, tardaba a veces hasta 30 segundos en arrancar un servidor local. Estudié su reemplazo: **Vite**. Esta nueva herramienta de empaquetado (Bundler), construida sobre el lenguaje Go y Esbuild, permitía compilar y arrancar el servidor de desarrollo en apenas 200 milisegundos. Además, implementaba un sistema de Recarga en Caliente (HMR) instantáneo. Adoptar Vite no era un capricho, era una necesidad para garantizar mi productividad durante las horas de programación.
- La revolución de Tailwind CSS v4:** La segunda gran revelación fue descubrir que Tailwind había lanzado recientemente su versión 4. Esta actualización mayor cambiaba drásticamente la arquitectura del framework. En versiones anteriores, era obligatorio mantener un archivo pesadísimo llamado `tailwind.config.js` para declarar colores y variables. La versión 4 adoptaba un motor de compilación Just-In-Time (JIT) mucho más agresivo y elegante, permitiendo inyectar variables nativas de CSS directamente mediante la directiva `@import "tailwindcss"`.

Conclusión y Sigüientes Pasos Esta semana de investigación teórica me salvó de cometer el grave error de iniciar un proyecto en 2026 con tecnologías de 2022. Comprender los nuevos motores de compilación de Vite y Tailwind me garantizó que

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

FITBOX nacería sobre cimientos tecnológicos de ultimísima generación, listos para soportar cualquier escala. Con las herramientas afiladas, solo faltaba tomar la última gran decisión sobre la base de datos en la nube.

17 | 23-enero: Análisis de Viabilidad de BaaS (Supabase vs Firebase)

Contexto y Punto de Partida La última pieza del puzzle arquitectónico antes de empezar a escribir código era la más importante: la persistencia de datos y la seguridad. En noviembre había decidido delegar la creación del servidor y de la base de datos a un proveedor Backend-as-a-Service (BaaS) para ahorrar tiempo de infraestructura y centrarme en aportar valor al gimnasio. En el mercado actual, existen dos titanes dominantes en este sector: Firebase (respaldado económicamente por Google) y Supabase (la alternativa Open Source). El objetivo de esta semana era realizar un análisis de viabilidad técnica y elegir al ganador.

Análisis, Reflexión y Toma de Decisiones Realicé simulaciones mentales de cómo se comportaría cada proveedor frente a los requisitos de FITBOX.

1. **El caso de Firebase:** Es el gigante de la industria. Ofrece una base de datos en tiempo real llamada Firestore. El problema radica en que Firestore es una base de datos NoSQL orientada a documentos. Como ya había analizado en octubre, el modelo de datos de un centro deportivo es puramente relacional. Si usara Firebase y el gimnasio quisiera un informe de "Todos los usuarios que asistieron a la clase de Boxeo de ayer y que tienen su cuota mensual impagada", tendría que descargar la colección completa de Clases, luego descargar las Reservas asociadas, luego los Usuarios y luego los Pagos, para finalmente cruzar todos

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

esos datos en la memoria del teléfono móvil. Esto es ineficiente, caro (Firebase cobra por cada lectura de documento) y muy lento.

2. **El caso de Supabase:** Esta plataforma se anuncia como "la alternativa Open Source a Firebase". Al leer su documentación técnica, la decisión se volvió evidente e instantánea. Bajo su moderna interfaz gráfica, Supabase aloja un motor de base de datos **PostgreSQL** empresarial. Esto encajaba como un guante con el Diagrama Entidad-Relación y el diseño SQL estricto que había desarrollado en los meses anteriores. Además, Supabase ofrecía ventajas críticas:

- Genera automáticamente una API RESTful instantánea a partir de las tablas creadas.
- Proporciona un módulo integrado de Autenticación (Supabase Auth) que gestiona de forma segura el registro de correos electrónicos, encriptación de contraseñas y emisión de Tokens JWT.
- Soporta RLS (Row Level Security), permitiendo bloquear el acceso a los datos directamente en el motor de la base de datos para prevenir hackeos.

¡Por supuesto! Aquí tienes la última y definitiva entrega de tu **Manual del Proyecto**. Con estas siete s finales, relataré la transición definitiva desde la teoría y la arquitectura hacia la programación real, el diseño estructurado y la conexión de la base de datos.

Al igual que las anteriores, cada está redactada con una profundidad analítica extrema, un tono académico y profesional, y una extensión pensada para ocupar una página completa en tu documento final.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

18 | 30-enero: Definición de la arquitectura de la aplicación y flujos de usuario (SPA)

Contexto y Punto de Partida Tras haber seleccionado a Supabase como el motor de nuestra base de datos, el trabajo de infraestructura en la nube estaba teóricamente resuelto. El enfoque debía regresar ahora a la capa visual: el Frontend. Sin embargo, no podíamos empezar a programar pantallas sueltas sin saber cómo se conectarían entre sí. FITBOX no es una web tradicional donde cada enlace carga un documento HTML nuevo; es una Single Page Application (SPA). En una SPA, el usuario descarga la aplicación una sola vez y, a partir de ahí, JavaScript se encarga de cambiar lo que se ve en pantalla de forma instantánea. Era obligatorio diseñar el mapa de carreteras, es decir, el enrutamiento y los flujos de seguridad del usuario.

Análisis, Reflexión y Toma de Decisiones Dediqué la semana a diseñar el árbol de navegación utilizando el estándar de la industria para React: React Router DOM. El reto mental no era simplemente decir "el botón A lleva a la pantalla B", sino anticiparme a los comportamientos anómalos o maliciosos de los usuarios.

Dividí la arquitectura de rutas en dos dominios estrictamente separados:

1. **Rutas Públicas:** Aquellas a las que cualquier persona en internet puede acceder. En el caso de FITBOX, se limitan a la pantalla principal de Inicio de Sesión (/login) y la pantalla de Recuperación de Contraseña.
2. **Rutas Privadas (Protegidas):** El corazón de la aplicación (/dashboard, /reservas, /socios, /maquinas).

El sentido común en ciberseguridad me exigía aplicar el principio de "Confianza Cero" (Zero Trust). Me planteé la siguiente situación: ¿Qué ocurre si un usuario anónimo, que

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

no ha puesto su correo ni contraseña, abre su navegador web y escribe manualmente la dirección `mipagina.com/dashboard?` Si la arquitectura es débil, el sistema podría cargar la pantalla e incluso mostrar datos sensibles o, en el mejor de los casos, bloquearse (crashear) por no encontrar el perfil del usuario. Para evitar esto, diseñé teóricamente un componente "Guardián" (AuthGuard o ProtectedRoute). La lógica matemática que programaría más adelante dictaría que, antes de renderizar (pintar) cualquier ruta privada, el enrutador debe detenerse una fracción de segundo, consultar a Supabase si existe un Token JWT válido y activo en el navegador del usuario. Si la respuesta es negativa, el Guardián intercepta la navegación y expulsa brutalmente al usuario de vuelta a la pantalla de /login.

Conclusión y Sigüientes Pasos Tener este mapa mental y teórico de las rutas de navegación era vital. Definir los flujos de entrada, salida y expulsión de usuarios garantizaba que, cuando llegara el momento de programar en React, el sistema no tendría "puertas traseras" ni fallos de lógica de navegación. La estructura de la SPA estaba blindada, y el siguiente paso lógico era definir cómo se iban a pintar esas pantallas.

19 | 06-febrero: Definición del sistema de diseño (Design System) y paleta de colores para Modo Oscuro

Contexto y Punto de Partida Un error devastador, pero extremadamente común entre los desarrolladores Junior, es comenzar a programar la interfaz escribiendo código CSS sobre la marcha. Si hoy decido que un botón es rojo y tiene bordes redondeados, y mañana programo otra pantalla y le pongo un rojo ligeramente distinto con bordes

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

cuadrados, el resultado final es un "diseño Frankenstein" que transmite poca profesionalidad. Para evitar esto, antes de tocar código, es obligatorio establecer un "Sistema de Diseño" (Design System). Una única fuente de la verdad para colores, tipografías y espaciados.

Análisis, Reflexión y Toma de Decisiones Basándome en los wireframes conceptuales que dibujé en octubre, me dediqué esta semana a estandarizar matemáticamente el diseño de FITBOX para que Tailwind CSS pudiera consumirlo de forma consistente.

El mayor reto era perfeccionar el **Modo Oscuro**. Hacer un buen modo oscuro no consiste simplemente en pintar el fondo de negro (#000000) y el texto de blanco (#ffffff). Ese contraste extremo crea un efecto de "halo" que cansa terriblemente la vista y dificulta la lectura continuada. Aplicando principios de ergonomía visual e interfaz de usuario (UI), estructuré la paleta de la siguiente manera:

- **Fondos base (Background):** Un color carbón muy oscuro, casi negro, con un ligero tinte azulado (#0f1115). Este tono da profundidad a la aplicación.
- **Superficies (Cards & Modals):** Las tarjetas que contienen información (como el perfil del usuario o un resumen de pago) deben "flotar" sobre el fondo. Les asigné un gris sutilmente más claro (#16181d) y un borde apenas perceptible (border-neutral-800).
- **Textos (Typography):** Nunca blanco puro. Se definió un gris muy claro (#e5e5e5) para los títulos principales y un gris medio (text-neutral-400) para los subtítulos, reduciendo la fatiga visual.
- **Color de Acento (Brand Color):** El famoso botón de "Llamada a la Acción" (Call to Action) se estandarizó en un rojo intenso (#dc2626), acompañado de sombras

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

dinámicas (shadow-red-500/30) que se activarían únicamente cuando el usuario pasara el dedo o el ratón por encima (estado hover/active).

Conclusión y Sigüientes Pasos El establecimiento de este Design System garantizaba que, sin importar cuántas pantallas diferentes programara en el futuro, todas compartirían un ADN visual idéntico. FITBOX se vería como una plataforma de gama alta, premium y cohesiva. El diseño estaba resuelto, así que el siguiente paso era preparar las carpetas físicas en el ordenador donde residiría este código.

20 | 13-febrero: Planificación de hitos de desarrollo y estructuración del repositorio

Contexto y Punto de Partida A medida que se acercaba el momento crítico de iniciar el proyecto y crear los primeros archivos, debía asegurar que el código no se convirtiera en un caos inmanejable. React es una librería "no opinada", lo que significa que no te obliga a organizar tus archivos de ninguna manera en particular. Esto es una ventaja para la libertad del programador, pero es una trampa mortal si no se tiene disciplina. Sin una arquitectura de carpetas sólida, el proyecto terminaría en el temido "código espagueti", donde buscar un error supone abrir veinte archivos diferentes.

Análisis, Reflexión y Toma de Decisiones Invertí esta semana en estudiar patrones de arquitectura de software para Frontend. Decidí adoptar una estructura modular estricta orientada a dominios y responsabilidades, separando drásticamente la lógica visual de la lógica de negocio.

Diseñé la siguiente arquitectura para la carpeta principal (/src) del repositorio:

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

- src/assets/: Directorio aislado para recursos estáticos pesados (el logotipo de FITBOX en PNG, iconos SVG).
- src/components/: El núcleo de la reutilización. Aquí programaría "piezas de Lego" independientes (por ejemplo, un componente <BotonRojo> o una <TarjetaDePago>) que podrían ser invocados decenas de veces en distintas pantallas sin tener que reescribir su código.
- src/pages/: Las vistas completas de la aplicación. Archivos grandes como Login.tsx o Dashboard.tsx, que actuarían como directores de orquesta, importando los componentes más pequeños y dándoles sentido.
- src/database/: Una carpeta sagrada y aislada. Aquí residiría el cliente de conexión con Supabase. Ningún componente visual debería hablar directamente con internet; todos deben pasar por los archivos de esta carpeta para garantizar el orden de las peticiones.
- src/layouts/: Archivos encargados de pintar las estructuras fijas (como el menú lateral que nunca desaparece) mientras el contenido interior cambia dinámicamente.

Conclusión y Sigüientes Pasos Esta estricta delimitación de responsabilidades aseguraba la escalabilidad del TFG. Si en el mes de mayo hubiera que cambiar cómo se ve un botón, solo tendría que ir al archivo src/components/Boton.tsx, y el cambio se propagaría mágicamente por todo el gimnasio. Con el "esqueleto" de carpetas diseñado en papel, quedaba un último cabo teórico por atar: la ciberseguridad a nivel de servidor.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

21 | 20-febrero: Planteamiento teórico de políticas de seguridad (RLS) para la base de datos

Contexto y Punto de Partida Antes de teclear la primera línea de código del producto final, era fundamental resolver el aspecto más crítico en cualquier aplicación comercial: la ciberseguridad. En el desarrollo web, existe un mandamiento inquebrantable: **"Jamás confíes en el cliente"**. El Frontend (la web que ve el usuario) vive en el navegador de su teléfono o su ordenador. Cualquier persona con conocimientos básicos puede abrir las "Herramientas de Desarrollador" (pulsando F12), manipular el código HTML o alterar las variables de JavaScript. Si la seguridad de FITBOX dependiera únicamente del Frontend, el sistema colapsaría en el primer intento de hackeo.

Análisis, Reflexión y Toma de Decisiones Sabiendo que el Frontend es vulnerable por naturaleza, la defensa debía construirse directamente en las trincheras del Backend, concretamente en el motor de la base de datos PostgreSQL de Supabase. Durante esta semana, estudié y diseñé teóricamente el paradigma **RLS (Row Level Security o Seguridad a Nivel de Fila)**.

Me planteé casos de uso maliciosos para diseñar las políticas de defensa: Escenario: Un socio del gimnasio accede a su perfil, donde aparece su "estado de pago". Abre la consola del navegador, intercepta la petición HTTP y envía al servidor un comando forzado diciendo: `UPDATE pagos SET estado = 'Pagado' WHERE id_usuario = mi_id`. Defensa: Si no hubiera RLS, la base de datos obedecería ciegamente y el socio se perdonaría su propia deuda. Sin embargo, diseñé la política de RLS con la siguiente premisa matemática: "PERMITIR UPDATE en la tabla 'pagos' SOLAMENTE SI el rol del usuario que ejecuta la petición es igual a 'Administrador'".

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

De este modo, aunque el atacante logre manipular la web y enviar la petición, cuando esta llega al servidor de Supabase, el motor de PostgreSQL lee el Token JWT oculto en la petición, detecta que el rol del atacante es "Socio", y rechaza la inserción en milisegundos devolviendo un error HTTP 403 (Prohibido).

Conclusión y Sigüientes Pasos Este planteamiento garantizaba que los datos financieros, los perfiles de los usuarios y las reservas de FITBOX estarían encriptados y protegidos a un nivel empresarial, haciendo que el sistema fuera prácticamente invulnerable a manipulaciones desde el cliente. Toda la arquitectura teórica, desde el diseño de pantallas hasta la ciberseguridad, estaba finalizada. Era el momento de encender los motores y preparar el ordenador físico para empezar a programar.

22 | 27-febrero: Preparación del entorno de desarrollo local (Node.js, VS Code, Git)

Contexto y Punto de Partida En el mundo del desarrollo existe una famosa (y temida) excusa: "Pues en mi ordenador sí que funcionaba". Para evitar problemas de versiones, dependencias rotas o fallos de compilación a la hora de defender el TFG, era obligatorio preparar un entorno de desarrollo profesional, limpio y estandarizado en la máquina física antes de empezar a descargar librerías. Una mesa de trabajo desordenada conduce inevitablemente a un producto defectuoso.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Análisis, Reflexión y Toma de Decisiones Dediqué la semana a acondicionar mi sistema operativo y mi IDE (Entorno de Desarrollo Integrado) como lo haría un desarrollador Senior al llegar a una nueva empresa.

1. **Entorno de Ejecución:** Verificamos e instalamos la versión LTS (Long Term Support) de **Node.js**. Elegir la versión de soporte a largo plazo garantiza que el proyecto no sufrirá roturas si Node publica actualizaciones experimentales a mitad de curso.
2. **Editor de Código:** Configuré **Visual Studio Code**. Un buen programador no escribe todo a mano; utiliza herramientas que le ayudan a no cometer errores. Instalé y configuré dos pilares fundamentales:
 - **ESLint:** Un linter estricto. Actúa como un profesor implacable que subraya en rojo el código si declaras una variable y no la usas, o si te equivocas en la sintaxis de TypeScript. Esto previene que errores minúsculos lleguen al navegador.
 - **Prettier:** Un formateador automático de código. Lo configuré para que, cada vez que yo pulse Ctrl + S para guardar, el editor ordene automáticamente la indentación, los espacios y las comillas. Esto asegura que los miles de líneas de código que formarán FITBOX parezcan escritas con una precisión robótica e impecable.
3. **Control de Versiones (Git):** Inicialicé Git en la carpeta de trabajo. El paso más crítico aquí fue crear el archivo .gitignore antes de hacer el primer guardado. Si no ignoraba carpetas masivas como node_modules o archivos sensibles como .env (donde más adelante pondría las contraseñas de la base de datos), corría el riesgo de subir accidentalmente mis claves privadas a internet, comprometiendo todo el proyecto en su primer día de vida.

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Conclusión y Sigüientes Pasos Con Node actualizado, el editor configurado para escribir código perfecto y Git vigilando cada cambio, la estación de trabajo física del desarrollador estaba operativamente impecable. Se acabaron los diagramas, los manuales teóricos y los diseños. El hito de la semana siguiente marcaría, por fin, la generación física de los archivos del proyecto.

23 | 06-marzo: Inicialización del proyecto base y configuración de empaquetadores

Contexto y Punto de Partida Llegó uno de los días más emocionantes del ciclo formativo: el momento de generar el andamiaje (scaffolding) de la aplicación y crear el repositorio físico. Este hito marcaría la transición del papel a los transistores. El objetivo de la semana era inicializar React, instalar el motor de diseño y comprobar que el entorno local era capaz de renderizar código en el navegador web sin errores.

Análisis, Reflexión y Toma de Decisiones Abrí la terminal de comandos de mi ordenador y ejecuté la instrucción que daría vida a FITBOX: `npm create vite@latest`. En el asistente de instalación, fui consecuente con las decisiones tomadas en el trimestre anterior y seleccioné explícitamente el framework **React** en combinación con el lenguaje **TypeScript**. Vite, haciendo gala de su extrema velocidad, descargó los paquetes base en cuestión de segundos.

Al entrar en la carpeta generada, lo primero que hice fue un ejercicio de limpieza profunda. Vite, por defecto, genera archivos de ejemplo (contadores interactivos, logos giratorios de React, estilos genéricos). Borré todo este código prefabricado, dejando el

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

archivo principal App.tsx y la carpeta src completamente vacíos. Quería empezar desde un lienzo en blanco puro.

A continuación, procedí a la instalación del motor gráfico: **Tailwind CSS v4**. Ejecuté los comandos de NPM necesarios para descargar sus dependencias. Siguiendo la nueva documentación de la versión 4, eliminé archivos de configuración antiguos y me dirigí al archivo de estilos global (src/styles/index.css). Allí, inyecté la directiva `@import "tailwindcss"` y apliqué las reglas del Modo Oscuro que había diseñado semanas atrás, forzando a que la etiqueta `<body>` del HTML adquiriera el color de fondo carbón (`#0f1115`).

Conclusión y Sigüientes Pasos Llegó el momento de la verdad. Ejecuté el comando `npm run dev` para arrancar el servidor de desarrollo local. Abrí mi navegador web, introduje la dirección `http://localhost:5173` y pulsé Enter. La pantalla ya no era el clásico fondo blanco de un navegador vacío, ni tampoco mostraba los ejemplos por defecto de React. Se mostró un lienzo negro profundo, limpio y elegante, listo para ser esculpido. El proyecto base había compilado perfectamente, certificando que todas las configuraciones previas eran correctas.

24 | 13-marzo: Conexión con Supabase, Auth, desarrollo del Login y enrutamiento base

Contexto y Punto de Partida Este hito marca la consolidación de meses de trabajo arquitectónico y el primer gran logro técnico de FITBOX. Tener un lienzo negro es un buen comienzo, pero el objetivo era dotar a la aplicación de vida, conectarla a la nube y crear la primera barrera de seguridad del sistema: La puerta de entrada (El Login). Si

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

este hito funcionaba, demostraría que el puente de comunicación entre mi Frontend en React y mi Backend en Supabase era sólido.

Análisis, Reflexión y Toma de Decisiones El trabajo se dividió en tres frentes de ataque simultáneos de alta complejidad técnica:

1. **La Conexión Segura (El puente al Backend):** Me dirigí al panel de control de Supabase en internet y creé el proyecto oficial "FITBOX". Extraje la URL del servidor y la API Key Anónima. Por sentido común en ciberseguridad, no pegué estas claves en mi código fuente. Las guardé en un archivo local .env, completamente oculto y protegido por Git. Acto seguido, programé el archivo Client.ts, encargado de leer esas claves ocultas e instanciar la conexión a la base de datos de forma global y segura.
2. **Maquetación de la Pantalla de Login (Login.tsx):** Traduje los bocetos de octubre a código React real. Utilizando las clases de utilidad de Tailwind (ej. bg-[#16181d] rounded-2xl shadow-xl), esculpí una tarjeta oscura en el centro de la pantalla. Importé el logotipo rojo de FITBOX, creé los campos de "Email" y "Contraseña", y un botón de acceso de color carmesí. La interfaz visual era imponente y premium.
3. **Lógica de Autenticación (Auth) y Enrutamiento:** Con la pantalla dibujada, vinculé los campos de texto a variables de estado de React (useState). Programé una función asíncrona que intercepta el clic del usuario y envía las credenciales cifradas por internet usando el método nativo supabase.auth.signInWithPassword(). Para culminar, configuré el BrowserRouter. Establecí la regla de que la ruta raíz (/) redirige automáticamente al Login. Implementé el código necesario para que, si Supabase responde que la contraseña es correcta, React ejecute el comando

	FITBOX	14/03/2026	MP
	MANUAL DE PROYECTO	PÁGINA 1/1	MP

Maps('/dashboard'), transportando al usuario hacia el interior de la aplicación sin recargar la página.

Conclusión y Sigüientes Pasos (El Hito Logrado) Al finalizar la jornada, realicé la prueba de fuego. Creé un usuario real con mi correo en la base de datos de Supabase. Introduje mis credenciales en la pantalla de localhost y pulsé el botón rojo. En una fracción de segundo, el sistema validó los datos en la nube, devolvió el Token de seguridad y la pantalla se transformó fluidamente en el esqueleto del Dashboard de FITBOX. El flujo cliente-servidor, la autenticación y la SPA funcionaban de manera impecable y profesional.